

C64uRemote für M5Stack Core

Technische Dokumentation

Version 1.0

Inhaltsverzeichnis

Überblick	2
Hardware	2
Zielformat	2
Peripherie	2
Programmierungsumgebung	3
PlatformIO (empfohlen)	3
Arduino IDE 2.x (Alternative)	3
Konfiguration (build_env.h)	3
WLAN-Subsystem	3
Laufzeitkonfiguration statt Compile-Zeit	3
Bildschirme und Bedienung	4
NVS-Layout	4
Kartenformat	4
/wifi.txt auf der SD-Karte	5
Setup-Portal	5
Direktmodus	5
Softwarearchitektur	6
Zustandsmodell	6
Rendering ohne PSRAM	6
Akkuanzeige	7
ReST-Anbindung an den C64 Ultimate	7
Verwendete Endpunkte	7
Abgewiesene Verbindungen	8
Eigener HTTP-Weg (seit v1.4.0)	8
NFC-Funkfeld	8
Wiederverbinden mit mehreren Netzen	8
CPU-Speed	9
Joystick-Ports	9
Disk-Images und Autostart	9
Streaming-Upload	9
NFC-Subsystem	9
Abfragestrategie	9
Warum eine eigene Probe	10
Rueckkehr zum Hauptbildschirm	10
Kartentypen	10
Kommandokarten	10
PowerOff mit Bestaetigung	11
Karten beschreiben	11
Datenformat (TeensyROM/Zaparoo-kompatibel)	11
NDEF-Parser: Toleranz	12
NFC-Info	12

Kopieren (Dump/Restore)	12
Bedienlogik	12
Tastenauswertung	12
PowerOff-Absicherung	13
Persistenz	13
Projektstruktur	13
Fehlerdiagnose	13
Quellen	13
Lizenz	14
Entstehung	14

Überblick

C64uRemote ist eine Firmware für den M5Stack Core (ESP32), die den Commodore 64 Ultimate bzw. Ultimate64 Elite-II über dessen ReST-API fernsteuert. Zusätzlich unterstützt sie einen RFID2-Leser (NXP WS1850S) und eine microSD-Karte, um Programme per NFC-Karte zu starten und Karten zu verwalten.

Die Firmware ist als einzelne Übersetzungseinheit (src/main.cpp, ca. 4200 Zeilen) umgesetzt und nutzt das Arduino-Framework für ESP32.

Grundlage: Originalprojekt von Karl Prosser (github.com/ReadyOS-C64/C64uRemote), Portierungen für M5StickC Plus2 und M5Dial von Martin Oswald (1MHz.de), diese Core-Fassung baut darauf auf.

Hardware

Zielplattform

Merkmal	Wert
Board	M5Stack Core Basic / Gray / Fire
SoC	ESP32 (Dual-Core, 240 MHz)
Display	320 × 240, ILI9342C, über M5GFX
Bedienung	3 Hardware-Tasten (A/B/C)
Speicher	Core Basic ohne PSRAM (~250 kB nutzbarer Heap)

Peripherie

Gerät	Anschluss	Details
Unit RFID2	Port A (Grove, rot)	I ² C, SDA = GPIO 21, SCL = GPIO 22, Adresse 0x28, 100 kHz
microSD	interner Slot	SPI, CS = GPIO 4, FAT32

Display und SD teilen sich den SPI-Bus. Die SD-Initialisierung versucht zuerst 20 MHz, bei Fehlschlag 4 MHz.

Programmierungsumgebung

PlatformIO (empfohlen)

Die Konfiguration liegt versioniert in `platformio.ini`, wodurch Builds reproduzierbar sind. Wesentliche Festlegungen:

Einstellung	Wert	Begründung
<code>board</code> <code>platform</code> <code>board_build.partitions</code>	<code>m5stack-core-esp32</code> <code>espressif32</code> <code>huge_app.csv</code>	Core Basic / Gray Arduino-Framework WiFi + SD + RFID überschreiten die Standard-App-Partition (1,25 MB)
<code>monitor_speed</code> <code>upload_speed</code>	115200 460800	serielle Ausgabe 921600 ist auf manchen USB-Serial-Chips unter macOS instabil

Bibliotheken (lib_deps):

- `m5stack/M5Unified` ($\geq 0.2.8$)
- `m5stack/M5GFX` ($\geq 0.2.11$)
- `bblanchon/ArduinoJson` ($\wedge 6.21.5$) — bewusst auf Version 6, da Version 7 `DynamicJsonDocument` nicht mehr kennt
- `MFRC522_I2C` von `kkloesener` (Git) — I²C-Treiber für den WS1850S

Ein zweites Environment `m5stack-fire` setzt zusätzlich `-DBOARD_HAS_PSRAM`.

Befehle:

```
pio run          # kompilieren
pio run -t upload # flashen
pio device monitor # serielle Ausgabe
pio run -e m5stack-fire -t upload
```

Arduino IDE 2.x (Alternative)

`main.cpp` in `C64uRemote.ino` umbenennen, die Bibliotheken manuell installieren und unter *Werkzeuge* → *Partition Scheme* **Huge APP** wählen.

Konfiguration (build_env.h)

`build_env.h` liefert nur noch die **Startwerte**. Sobald im NVS eine Netzkonfiguration steht, wird die Datei ignoriert. Vorlage `build_env.h.example` nach `build_env.h` kopieren und ausfüllen:

```
#define C64U_WIFI_SSID      "MeinWLAN"
#define C64U_WIFI_PASSWORD "MeinPasswort"
#define C64U_TARGET_HOST   "192.168.0.64"
#define C64U_TARGET_PASSWORD ""           // nur falls im C64 gesetzt
```

Fehlt die Datei, kompiliert das Projekt mit leeren Defaults; der Core zeigt beim Start dann *SETUP* > *WLAN* und lässt sich am Gerät einrichten.

WLAN-Subsystem

Laufzeitkonfiguration statt Compile-Zeit

SSID, Passwort, `c64u`-Adresse und `c64u`-Passwort liegen zur Laufzeit im NVS und sind über *SETUP* → *WLAN* änderbar. Bis zu `kWiFiProfileMax` (= 4) Profile werden vorgehalten:

```

struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];
size_t      gWifiCount;
size_t      gWifiTry;      // Profil fuer den naechsten Verbindungsversuch

```

beginWifi() nimmt gWifiProfiles[gWifiTry] und rückt den Index anschließend um eins weiter. Schlägt eine Verbindung fehl, probiert serviceWifi() nach kWifiRetryMs das nächste Profil – über mehrere Durchläufe wandert der Versuch so durch alle bekannten Netze. Beim Start sucht wifiPickBestProfile() einmal die Umgebung ab und startet mit dem stärksten bekannten Netz.

wifiAddProfile() sortiert ein neues Netz vorn ein; ein bereits bekanntes Netz bekommt nur ein neues Passwort und rutscht ebenfalls nach vorn. Das älteste Profil fällt bei Bedarf hinten heraus.

Bildschirme und Bedienung

Fünf Bildschirme kommen dazu. Bedient werden sie wie alle Listenseiten: **A** und **C** blättern, **B** löst aus, **B lang** führt zurück.

Bildschirm	Aufgabe
WifiMenu	Untermenü mit den acht Einträgen
WifiScan	Liste der gefundenen Netze
WifiCard	Karte auflegen, Passwort einlesen
WifiPortal	Accesspoint läuft, zeigt SSID, Passwort, IP und Restzeit
WifiSaved	Gespeicherte Netze: verbinden, löschen oder auf Karte schreiben

Die Einstellungsliste besitzt jetzt ein enum SettingsId mit kSetWifi als letzter Aktion. kSetLastAction trennt Aktionen von Schaltern, ein static_assert hält Namen und Beschriftungen zusammen.

NVS-Layout

Namensraum c64unet, getrennt von den Bedieneinstellungen in c64uremote:

Schlüssel	Inhalt
wn	Anzahl der Profile (0...4)
s0...s3	SSID
p0...p3	Passwort
host	Adresse des c64u
hpass	Passwort des c64u
dmode	Direktmodus an/aus
dssid, dpass	SSID und Passwort des Direktnetzes
dnet	Adressbereich des Direktnetzes, z. B. 192.168.4

Factory Reset betrifft nur c64uremote; die Netzkonfiguration bleibt erhalten und wird ausschließlich über *WLAN* → *Alle löschen* verworfen.

Kartenformat

WLAN-Karten benutzen dasselbe Schema wie WLAN-QR-Codes, gespeichert als gewöhnlicher NDEF-Textrecord:

```
WIFI:S:<ssid>;T:WPA;P:<passwort>;;
```

parseWifiText() wertet die Felder S und P aus, akzeptiert mit Backslash maskierte Sonderzeichen und versteht zusätzlich die Kurzform WIFI:<ssid>;<passwort>. wifiCardText() ist das Gegenstück und wird von *WLAN* → *Auf NFC-Karte* benutzt.

Auf der Einrichtungsseite (ScreenMode::WifiCard) gilt ein Kartentext ohne WIFI:-Präfix als reines Passwort für das vorher gewählte Netz.

Außerhalb der Einrichtung erkennt `processCard()` eine vollständige WIFI:-Karte, speichert das Netz und verbindet sofort. Dabei wird `gWifiTry` gezielt auf dieses Profil gesetzt und bis zu `kWifiCardConnectMs` (8 s) auf `WL_CONNECTED` gewartet; danach übernimmt wieder `serviceWifi()`. Ist das Netz bereits verbunden, bricht die Funktion mit *SCHON VERBUNDEN* ab – das verhindert eine Endlosschleife, wenn die Karte liegen bleibt und die Hintergrundabfrage sie erneut erkennt.

/wifi.txt auf der SD-Karte

`loadWifiFromSd()` liest ein einfaches Schlüssel-Wert-Format. Jede neue `ssid`-Zeile beginnt einen Eintrag; `#` und `;` leiten Kommentare ein. Zusätzlich werden `host` und `hostpass` erkannt.

```
ssid = MeinWLAN
pass = geheim
```

```
host      = 192.168.0.64
hostpass =
```

Gelesen wird die Datei beim Start (nur solange `gWifiCount == 0`) und auf Anforderung über *WLAN* → *Von SD laden*.

`saveWifiToSd()` ist das Gegenstück: Es schreibt alle Profile samt `host` und `hostpass` mit einem Kommentarkopf zurück nach `/wifi.txt`. Eine vorhandene Datei wandert vorher per `SD.rename()` nach `/wifi.bak`; schlägt das fehl, wird sie gelöscht. Die Funktion liefert die Anzahl der geschriebenen Netze und setzt bei Fehlern einen Klartexthinweis in `errorOut`.

Setup-Portal

`startPortal()` schaltet auf `WIFI_AP` um, öffnet einen Accesspoint auf festem Kanal 1 (`kPortalSsid` / `kPortalPass`), startet einen `DNSServer` als Captive-Portal-Umleitung und einen `WebServer` mit zwei Routen (`/` und `/save`). Der Station-Teil wird bewusst abgeschaltet: bliebe er aktiv, würde er im Hintergrund weiter nach dem gespeicherten Netz suchen, dabei den Funkkanal wechseln und angemeldete Clients abwerfen.

`servicePortal()` läuft in jeder Schleife mit, hält die Leerlaufuhr an, solange ein Client verbunden ist, und beendet das Portal nach `kPortalIdleMs` (5 min) oder `kPortalCloseMs` nach einem erfolgreichen Speichern. `stopPortal()` stellt `WIFI_STA` wieder her und verbindet sofort neu.

Beide Klassen kommen aus dem Arduino-ESP32-Kern (`WebServer.h`, `DNSServer.h`); zusätzliche Bibliotheken sind nicht nötig.

Direktmodus

Für Treffen ohne Router spannt der Core ein eigenes WLAN auf (`WIFI_AP`, ohne Station-Teil – aus demselben Grund wie beim Setup-Portal: eine suchende STA wechselt den Kanal und wirft angemeldete Geräte hinaus).

SSID / Passwort	C64uRemote-Direct / c64ultimate (Vorgaben, änderbar)
Kanal, max. Clients	6, 8
Adresse Core	<netz>.1
DHCP-Bereich	<netz>.64 ... <netz>.74
Ziel c64u	<netz>.64
<netz>	192.168.4 (Vorgabe) oder 192.168.2, per <code>direct_net</code> beliebig

Reihenfolge beim Start: erst `WiFi.softAP()`, dann `WiFi.softAPConfig(ip, ip, 255.255.255.0, <netz>.64)`. Der vierte Parameter setzt den Anfang des DHCP-Bereichs (Arduino-Core 2.0.17 vergibt ab dort elf Adressen). Andersherum kann je nach Core-Version die Vorgabe 192.168.4.1 stehen bleiben – beim Setup-Portal fällt das nicht auf, weil es genau diese Adresse benutzt.

Zieladresse: `targetHost()` liefert im Direktmodus `gDirectHost` statt der gespeicherten Heimadresse, die dadurch unangetastet bleibt. Ist das Gerät als normaler Client mit einem Netz

verbunden, dessen SSID der des Direktnetzes entspricht, gilt ebenfalls <netz>.64 – so kann ein zweiter Fernbediener mitlaufen, ohne dass seine Einstellungen geändert werden.

c64u finden: Die erste DHCP-Vergabe ist .64. War ein anderes Gerät schneller, sucht `directNextCandidate()` weiter: Kandidaten sind .64 und alle Adressen aus `esp_wifi_ap_get_sta_list()` + `esp_netif_get_sta_list()`. Nach jedem gescheiterten Statustest wird der nächste versucht. Als c64u gilt, wer auf /v1/version mit JSON antwortet (oder mit 401/403, wenn ein Passwort fehlt). Hat eine Adresse schon einmal geantwortet, wird erst nach dem zweiten Fehlschlag in Folge gewechselt – der c64u weist gelegentlich von sich aus ab (siehe *Abgewiesene Verbindungen*).

Statustest: Ist niemand angemeldet, entfällt der Test ganz. Kommt ein Gerät hinzu (`WiFi.softAPgetStationNum()` steigt) und ist der c64u noch nicht bestätigt, wird sofort geprüft, wieder beginnend bei .64. Weil der c64u dann oft noch mit DHCP beschäftigt ist, folgen bis zu sechs Nachprüfungen im Abstand von 3 s (`directRecheckDue()`), auf jeder Seite, bis er antwortet. Im Direktnetz gilt für den Verbindungsaufbau außerdem eine Frist von 1 s.

DHCP zurücksetzen: Der DHCP-Server des ESP32 vergibt Adressen über einen Zeiger, der nur vorwärts läuft. Gibt der c64u beim Abmelden seine Adresse frei oder fragt er beim Wiederkommen nach der alten, verliert der Server den Eintrag und vergibt die nächste (.65, .66 ...). Deshalb startet `directRestartDhcp()` den Server neu, sobald sich das letzte Gerät abgemeldet hat – danach beginnt er wieder bei .64. Mit weiteren angemeldeten Geräten unterbleibt das, weil der Server dabei alle Zuteilungen vergisst und sonst eine Adresse doppelt vergeben könnte; dann findet die Suche den c64u unter seiner neuen Adresse.

Umschalten: `setDirectMode()` speichert, verwirft den Verbindungsstatus und startet AP bzw. WLAN neu. Ein gezielt gewähltes Netz (Liste, Suche, WLAN-Karte) beendet den Direktmodus. Das Setup-Portal verdrängt den Direkt-AP vorübergehend; nach dem Portal startet `beginWiFi()` ihn wieder.

Suche im Direktmodus: Scannen kann nur der Station-Teil. Für die Dauer der Suche wird deshalb kurz auf `WIFI_AP_STA` und danach zurück auf `WIFI_AP` geschaltet.

NVS (Namensraum `c64unet`): `dmode` (bool), `dssid`, `dpass`, `dnet`. Unbrauchbare Werte (SSID leer oder > 32 Zeichen, Passwort < 8 oder > 63, Adressbereich nicht drei Zahlen 0...255) werden beim Laden durch die Vorgaben ersetzt.

Softwarearchitektur

Zustandsmodell

Der gesamte Laufzeitzustand liegt im globalen struct `AppState app`. Die Anzeige folgt einem Bildschirm-Enum:

```
enum class ScreenMode {
    Home, CpuMenu, Status, Settings, SdBrowser,
    RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore, Busy
};
```

Die Hauptschleife `loop()` arbeitet bei ~30 fps (`kFrameMs = 33`):

1. `M5.update()` – Tasten und Timer aktualisieren
2. `serviceWiFi()` / `refreshConnectionStatus()` – Netzwerk pflegen
3. `handleButtons()` – Tasteneingaben auswerten
4. `serviceRfid()` – RFID-Polling (alle 250 ms, nur auf RFID-Screens)
5. `updateHomeDemo()` – Animationsablauf
6. `render()` – Anzeige aktualisieren

Rendering ohne PSRAM

Der Core Basic hat kein PSRAM; drei Vollbild-Puffer ($3 \times 320 \times 240 \times 2 \text{ Byte} \approx 460 \text{ kB}$) passen nicht in den Heap. Die StickC-Fassung nutzt solche Puffer – diese Portierung nicht.

Stattdessen wird **zeilenweise direkt ins Display gezeichnet**. Das Logo liegt im Flash (`1MHz_logo_rgb565.h`, 240×135 , RGB565) und wird per `pgm_read_word` gelesen. Zusätzlicher

RAM-Bedarf: ein Zeilenpuffer (rowBuf, 640 Byte) und zwei Skalierungstabellen (logoXMap, logoYMap, zusammen ~1 kB).

Die Effektberechnung (drawDistortedRows, drawRotoZoom, drawRipple, drawRasterBars) skaliert über FX Detail: bei „Half“ wird nur jeder zweite Pixel und jede zweite Zeile gerechnet und beim Ausgeben verdoppelt (Faktor 4 weniger Rechenlast). Menüs werden nur bei Änderung neu gezeichnet (screenDirty, barDirty), damit nichts flackert.

Akkuanzeige

Der Ladestand kommt von M5Unified: M5.Power.getBatteryLevel() (0-100, negativ = kein Akku), M5.Power.isCharging() und M5.Power.getVBUSVoltage(). Gelesen wird höchstens alle kBattPollMs (5 s); Blinken und Wechsel laufen auf dem gemerkten Wert, kosten also keine weiteren I²C-Zugriffe.

Im Core sitzt ein **IP5306**. getBatteryLevel() liefert dort nur 0, 25, 50, 75 oder 100, und getVBUSVoltage() kennt er nicht (Rückgabe -1) – „am Strom“ heißt hier deshalb schlicht „lädt gerade“.

Gezeichnet wird an drei Stellen:

- drawBatteryLine() ersetzt den Strich unter der Leiste. Die Grundlinie bleibt kColLine, der gefüllte Teil ist zwei Pixel hoch. Bei 0 % wäre er zu schmal zum Sehen, deshalb if (width < 4) width = 4;.
- drawBatterySymbol() zeichnet Rahmen (32 × 12 px) und Pluspol und setzt die Prozentzahl mittig hinein. Gefüllt wird der Rahmen bewusst nicht – das erledigt der Strich, und die Zahl bleibt lesbar.
- drawChargeBolt() setzt links davon einen 5 × 7 Pixel großen Blitz, Zeile für Zeile aus einer kleinen Tabelle statt aus Dreiecken; bei dieser Größe stimmt die Form sonst nicht.

Die Farbe für alle drei liefert batteryColor(), die Schwellen stehen als kBattGreen, kBattYellow und kBattBlinkAt oben in der Datei.

drawStatusBar() lief bisher höchstens einmal pro Sekunde – zu selten für ein sichtbares Blinken. In der untersten Stufe verkürzt render() das Intervall deshalb auf kBattBlinkMs (500 ms).

ReST-Anbindung an den C64 Ultimate

Alle Kommandos laufen über die HTTP-ReST-API der Ultimate-Firmware (ab 3.11). Basis-URL: `http://<host>/v1/...`. Ist ein Netzwerkpasswort gesetzt, wird es im Header X-Password mitgesendet.

Verwendete Endpunkte

- **Version/Erreichbarkeit** — GET /v1/version
- **Reset** — PUT /v1/machine:reset
- **Reboot** — PUT /v1/machine:reboot
- **Ausschalten** — PUT /v1/machine:poweroff
- **Ultimate-Menü** — PUT /v1/machine:menu_button
- **Speicher schreiben** — PUT/POST /v1/machine:writemem?address=...
- **Speicher lesen** — GET /v1/machine:readmem?address=...&length=...
- **Konfig-Kategorien** — GET /v1/configs
- **Konfig lesen/setzen** — GET/PUT /v1/configs/<Kategorie>/<Item>
- **PRG starten** — POST /v1/runners:run_prg
- **CRT starten** — POST /v1/runners:run_crt
- **SID/MOD spielen** — POST /v1/runners:sidplay / :modplay
- **Disk einlegen** — POST /v1/drives/<a|b>:mount?type=...&mode=...
- **Laufwerk an** — PUT /v1/drives/<a|b>:on

sendApiRequest() kapselt GET/PUT über einen eigenen HTTP-Weg (siehe unten) und wertet die JSON-Antwort mit ArduinoJson aus (errors-Array).

Abgewiesene Verbindungen

Der HTTP-Server der Ultimate-Firmware nimmt jeweils nur eine Verbindung an und weist weitere mit einem TCP-RST ab; HTTPClient meldet das als *connection refused*. Beobachtet wurde das auch ohne ein zweites Gerät im Netz – es tritt also sporadisch auf und ist kein Funk- oder Adressproblem.

Der eigentliche Aufruf steckt in `sendApiRequestOnce()`. `sendApiRequest()` ist ein Mantel darum: Kommt keine Verbindung zustande (`HTTPC_ERROR_CONNECTION_REFUSED`), folgen nach `kApiRetryDelayMs` bis zu zwei weitere Versuche (`kApiConnectAttempts = 3`). Wiederholt wird **ausschließlich** in diesem Fall – dann ist beim c64u nichts angekommen und ein Befehl kann sich nicht doppelten. Lesefehler und HTTP-Fehlerstatus (4xx, 5xx) werden unverändert durchgereicht.

Zusätzlich spart `refreshConnectionStatus()` eine Anfrage: Ohne hinterlegtes Passwort wäre die zweite Abfrage byte-gleich mit der ersten, weil der Header X-Password nur gesetzt wird, wenn überhaupt eines da ist. Das halbiert die Grundlast auf dem c64u.

Eigener HTTP-Weg (seit v1.4.0)

Alle Anfragen an den c64u – `sendApiRequestOnce()`, `readC64Byte()` und die Uploads in `uploadFile()` – laufen nicht mehr über HTTPClient/WiFiClient, sondern über kleine Funktionen auf lwIP-Sockets:

- `rawOpen()` baut die Verbindung nicht blockierend auf und schaut per `select()` ohne Wartezeit alle 2 ms nach; Frist `kHttpConnectTimeoutMs` (1,5 s), im Direktmodus `kHttpConnectDirectMs` (1 s).
- `rawSendAll()` schickt Anfrage bzw. Datei in Stücken mit `MSG_DONTWAIT`.
- `rawReadResponse()` liest die Antwort, bis der c64u die Verbindung schließt (oder kurz nach dem letzten per Content-Length angekündigten Byte), und wertet Statuszeile, Content-Length und chunked aus. Schließt der c64u ohne Antwort, lautet der Fehler *connection lost*, bleibt die Antwort aus, *read Timeout*.

`rawHttpRequest()` fasst das für GET/PUT zusammen; jede Anfrage trägt `Connection: close`. Hintergrund: Mit HTTPClient gingen im Heimnetz zeitweise die meisten Abfragen verloren, während eine ohne Warten abgefragte Verbindung zuverlässig durchkam.

`staSsid()` und `staRssi()` liefern Netzname und Signalstärke aus einem Zwischenspeicher, der höchstens einmal pro Sekunde über `esp_wifi_sta_get_ap_info()` erneuert wird. `WiFi.SSID()` fragt bei jedem Aufruf den Treiber, und `targetHost()` braucht den Netznamen ständig (Erkennung des Direktnetzes als Client). `directStationCount()` erfragt die Zahl der Stationen im Direktmodus höchstens alle 500 ms.

NFC-Funkfeld

Der MFRC522-Treiber lässt das 13,56-MHz-Feld nach `PCD_Init()` dauerhaft an. `rfidFieldOn()` und `rfidFieldOff()` schalten es nur für die Kartenprobe (`cardPresent()`, `cardPresentQuick()`, danach 5 ms Anlaufzeit für die Karte) und für die Bearbeitung einer erkannten Karte ein; bei einer leeren Probe geht es wieder aus. Vor jeder Netzwerkanfrage (`rawOpen()`) wird es ebenfalls abgeschaltet – laut M5Stack teilen sich beim M5Dial RFID und WLAN die Antenne, und das WLAN ist blockiert, solange das Feld an ist. Damit eine liegen gebliebene Karte nicht bei jedem Einschalten des Feldes erneut ausgeführt bzw. beschrieben wird, merkt sich `rfidHoldCard()` nach `processCard()` ihre UID. `rfidHeldCardGone()` schaltet bei jeder Abfrage das Feld kurz ein, liest die UID und schaltet sofort wieder ab; dieselbe Karte wird ignoriert, erst zwei Fehlanzeigen in Folge oder eine andere Karte geben den Weg frei. Beim M5Dial, dessen NFC-Antenne direkt neben der WLAN-Antenne liegt, störte das dauerhaft eingeschaltete Feld den WLAN-Empfang; hier spart das Abschalten vor allem Strom.

Wiederverbinden mit mehreren Netzen

`beginWiFi()` schaltet nach jedem Versuch auf das nächste gespeicherte Profil weiter. Ohne Gegenmaßnahme heißt das: Sind zwei Netze hinterlegt und nur eines ist erreichbar, trifft es nach

einem Aussetzer jedes zweite Mal das tote Netz und kostet einen kompletten Wiederholungstakt (kWiFiRetryMs, 10 s).

serviceWiFi() merkt sich deshalb beim Verbinden über wifiProfileIndex(staSsid()) das Profil, mit dem es geklappt hat, und legt es als nächsten Versuch fest; gWifiNoted sorgt dafür, dass das nur einmal je Verbindung passiert. Nach einem Aussetzer geht der erste Versuch damit wieder an das funktionierende Netz, das tote kommt nur dran, wenn das gute wirklich weg ist.

CPU-Speed

Der Pfad zum CPU-Speed-Item ist nicht fest, sondern wird gesucht: zuerst in „U64 Specific Settings“, sonst über alle Kategorien (resolveCpuPath). Die verfügbaren Werte kommen aus dem values-Array des Items; als Rückfall dient eine feste Liste (setFallbackCpuChoices).

Joystick-Ports

Für das Tauschen der Joystickports gibt es keinen machine:-Befehl. Die Belegung ist ein Konfigurationseintrag, im Test *Joystick Swapper* in der Kategorie *U64 Specific Settings* mit den Werten Normal, Swapped, WASD Port 2 und WASD Port 1. Gesetzt wird sie deshalb über `/v1/configs/<Kategorie>/<Eintrag>?value=...`, genau wie die Taktstufe.

resolveJoyPath() sucht wie beim Takt zuerst in *U64 Specific Settings* und geht sonst alle Kategorien durch, bis ein Eintragsname „Joystick“ enthält; ein Umbenennen durch eine spätere Firmware fällt damit nicht auf. refreshJoyChoices() liest values und current.

joyTokenFromValue() und joyValueFromToken() rechnen zwischen Gerätwert und Kartenkürzel um (WASD Port 1 <-> WASD1), damit auf einer Karte kein Leerzeichen und keine firmwarespezifische Schreibweise stehen muss. toggleJoystickSwap() schaltet zwischen Normal und Swapped um und landet aus einem WASD-Modus wieder auf Normal; cycleJoystickValue() geht im Setup der Reihe nach durch alle gemeldeten Werte.

Disk-Images und Autostart

Beim Mounten wird das Ziellaufwerk ermittelt (resolveTargetDrive): bei „Auto“ sucht der Core über GET `/v1/drives` das Laufwerk mit bus_id: 8. Nach dem Mounten folgt :on, dann je nach *Disk Action* ein Reset und ein Autostart.

Der Autostart tippt per DMA in den C64-Tastaturpuffer (writemem auf \$0277 ff., Zählregister \$C6). Verwendet werden die abgekürzten BASIC-Befehle `l0"*"`, `8,1` und `rU`, damit die Zeile in die zehn Byte des Puffers passt.

Die Wartezeiten sind **nicht fest**, sondern über \$CC (BLNSW, Cursor-Blinken) geregelt: waitCursorBlinking() pollt readmem und erkennt am Blinken, wann BASIC bereit ist bzw. wann das Laden abgeschlossen ist. Timeout fürs Laden: 3 Minuten.

Streaming-Upload

Da große .d64 (bis ~800 kB bei .d81) nicht in den RAM passen, sendet uploadFile() die Datei blockweise (1 kB) direkt aus dem SD-Stream in einen WiFiClient-Socket. Multipart-Grenzen und Header werden manuell erzeugt; die Argumente type/mode stehen in der URL, nicht als Formularfeld (die Firmware interpretiert sonst jedes Multipart-Teil als Datei).

NFC-Subsystem

Abfragestrategie

serviceRfid() arbeitet in zwei Betriebsarten:

1. **Auf den RFID-Seiten** (RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore) wird alle 250 ms mit cardPresent() voll abgefragt.

2. **Auf dem Hauptbildschirm** laeuft je nach Einstellung *Auto-NFC* eine schnelle Probe mit `cardPresentQuick()`. Wird dabei eine Karte erkannt, setzt der Code `autoRfidActive`, wechselt per `setScreen()` auf `RfidRun`, zeichnet einmal `render()` und ruft dieselbe Verarbeitung auf wie die Leseseite.

Die eigentliche Kartenbehandlung steckt in `processCard()`. Beide Pfade rufen sie auf, es gibt also nur eine Implementierung.

Warum eine eigene Probe

Liegt keine Karte auf, wartet der MFRC522 nach dem REQA-Kommando, bis sein interner Timer ablaeuft. `PCD_Init()` stellt da fuer `0x03E8 = 1000` Schritte zu je `25 us` ein, also `25 ms`. Genau so lange steht die Hauptschleife, was bei laufender Animation als Ruckler sichtbar waere.

Eine Karte antwortet jedoch weit schneller: die Frame Delay Time betraegt bei `106 kBit/s` etwa `86 us`. `cardPresentQuick()` verkuerzt das Zeitfenster deshalb nur fuer die Probe auf rund `2 ms` und stellt es unmittelbar danach wieder her - noch vor `PICC_ReadCardSerial()`. Auswahl, Authentifizierung und alle Schreibvorgaenge laufen damit unveraendert mit dem vollen Zeitfenster.

```
void setRfidTimerReload(uint16_t ticks) {           // 1 Tick = 25 us
    rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegH, ticks >> 8);
    rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegL, ticks & 0xFF);
}
```

Der Ausgangswert wird nicht fest verdrahtet, sondern in `initRfid()` aus `TReloadRegH/L` gelesen und in `gRfidTimerReload` gemerkt. Aendert eine kuenftige Bibliotheksversion die Voreinstellung, bleibt das Verhalten korrekt.

Kosten. Eine ergebnislose Probe besteht aus wenigen I2C-Registerzugriffen plus den etwa `2 ms` Wartezeit, zusammen grob `5 ms`. Bei der Voreinstellung von `700 ms` Abstand ergibt das eine Grundlast von unter einem Prozent; ein Frame von `33 ms` wird dadurch nicht verfehlt.

<i>Auto-NFC</i>	Abstand	ungefaehre Grundlast
Off	-	0 %
1.5s	1500 ms	~0,3 %
0.7s	700 ms	~0,7 %
0.3s	300 ms	~1,7 %

Rueckkehr zum Hauptbildschirm

Eine automatisch geoeffnete Leseseite faellt nach `kAutoRfidHoldMs` (`20 s`) von selbst zurueck; so lange kann man weitere Karten auflegen. Das Flag `app.autoRfidActive` unterscheidet dabei die automatisch geoeffnete Seite von einer, die der Benutzer selbst angewaehlt hat - letztere bleibt stehen. Jeder Tastendruck und jede eigene Navigation loeschen das Flag.

Kartentypen

Familie	Erkennung	Zugriff
MIFARE Classic 1K/4K/Mini	SAK	16-Byte-Blöcke, Authentifizierung nötig
NTAG213/215/216, Ultralight	SAK 0x00	4-Byte-Seiten, kein Schlüssel

`cardKind()` unterscheidet anhand des SAK-Bytes. Der genaue NTAG-Typ kommt aus dem `GET_VERSION`-Kommando (`0x60`), ersatzweise aus dem Capability Container (Seite 3).

Kommandokarten

Traegt eine Karte statt eines Dateipfads das Praefix `CMD:`, wird der Inhalt als Befehl fuer den `c64u` ausgefuehrt. Weder SD-Karte noch Datei sind da fuer noetig.

```

CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      sofort ausschalten
CMD:POWEROFF=8      nachfragen, 8 s Bestaetigungsfenster
CMD:POWEROFF        nachfragen mit der Geraeteeinstellung "NFC-Cmd PowOff"
CMD:M5OFF           den Core selbst ausschalten (auch CMD:DIALOFF)
CMD:CPU=10          CPU auf 10 MHz
CMD:JOY             Joystickports umschalten (Normal <-> Swapped)
CMD:JOY=SWAPPED     Ports fest setzen; auch NORMAL, WASD1, WASD2

```

parseCardCommand() zerlegt den Text: Praefix pruefen, optionales Argument hinter = abtrennen, Schluesselwort in Grossbuchstaben vergleichen. Leerzeichen und Gross-/Kleinschreibung sind egal. Der Inhalt bleibt ein gewoehnlicher NDEF-Textrecord, jede NFC-App kann so eine Karte lesen und schreiben.

processCard() prueft im Lesezweig zuerst auf einen Befehl und ruft runCardCommand() auf. Traegt eine Karte zwar das Praefix, aber kein bekanntes Schluesselwort, meldet das Geraet *BEFEHL UNBEKANNT*, statt einen Dateipfad daraus zu machen.

PowerOff mit Bestaetigung

Die Wartezeit steht als Argument **auf der Karte**, nicht im Geraet; cardPowerOffSeconds() liefert sie zurueck. Ohne Argument gilt die Einstellung *NFC-Cmd PowOff* (3/5/8/15 s), 0 bedeutet "ohne Nachfrage".

Der Ablauf laeuft ueber drei Felder in AppState:

```

bool      cardPowerOffPending;
String    cardPowerOffUid;      // nur dieselbe Karte bestaetigt
uint32_t  cardPowerOffUntilMs;

```

Bestaetigt wird durch erneutes Auflegen derselben Karte oder durch einen Tastendruck. Eine andere Karte bricht ab, ebenso das Ablaufen des Fensters - beides loescht das Flag, ohne etwas auszuloesen. Die Bindung an die UID verhindert, dass eine zufaellig danebengelegte Karte den Rechner ausschaltet.

Karten beschreiben

cmdListAt() baut die Auswahlliste: fuenf feste Befehle, danach alle CPU-Stufen, die app.cpuDisplayOptions gerade enthaelt (aus dem c64u geladen, sonst die eingebaute Ersatzliste). cardCommandText() erzeugt daraus den Kartentext. Der Bildschirm ScreenMode::CmdPick zeigt die Liste, die Auswahl landet in app.pendingCardText und wird von der Schreibseite bevorzugt vor pathToCardText(app.pendingPath) verwendet.

Datenformat (TeensyROM/Zaparoo-kompatibel)

Auf der Karte liegt ein einzelner **NDEF-Record vom Typ Text** (Well Known, UTF-8). Inhalt ist der Pfad zur Programmdatei:

```
SD:OneLoad v5/Bubble Bobble.crt
```

Präfixe SD:, USB:, TR: werden akzeptiert (letzttere zwei werden auf der SD gesucht). Ein ? als Dateiname bzw. ein Verzeichnispfad startet eine zufällige Datei aus dem Ordner. Maximale Textlänge: 246 Zeichen.

Ablage:

- NTAG/Ultralight: NDEF-TLV ab Seite 4, Seiten 0–3 bleiben unberührt.
- MIFARE Classic: NDEF-TLV in den Datenblöcken ab Block 4, Trailer werden übersprungen. Authentifizierung zuerst mit NDEF-Schlüssel D3F7D3F7D3F7, sonst Werksschlüssel FFFFFFFF.

NDEF-Parser: Toleranz

`parseNdefText()` liest den Text robust aus. Wichtige Sonderfälle:

- **Falsche Payload-Länge:** Der TeensyROM schreibt konstant 0x10 in das Längenbyte, obwohl die TLV-Länge korrekt ist. Beim letzten Record (ME-Flag) hat daher die TLV-Länge Vorrang, sonst würde der Pfad abgeschnitten.
- Füllbytes (0x00) und der TLV-Terminator (0xFE) beenden den Text.
- Mehrfache Schrägstriche (SD://Ordner) werden zusammengefasst.
- Vorangestellte Lock-Control-TLVs und 3-Byte-Längen werden übersprungen.

Zum Schreiben erzeugt `buildNdefText()` einen sauberen kurzen Record. Das frühere Rohformat C64UPATH wird beim Lesen weiterhin erkannt.

NFC-Info

`collectCardInfo()` füllt zwei Seiten (im Gerät mit A/C umschaltbar):

- **Seite 1:** UID, SAK, Typ, Speichergröße, Version/Hersteller, Inhalt, Pfad, Dateiname und Abgleich gegen die SD-Karte.
- **Seite 2:** Hex-Dump der Seiten 0-15, Lock-Bytes, ausgewerteter Capability Container, Passwortschutz aus der Konfigurationsseite und der NFC-Lesezähler.

Befehle, die nicht jede Karte kennt (`GET_VERSION`, Konfigseite, `READ_CNT`), laufen bewusst zuletzt, da ein nicht unterstützter Befehl die Karte deselektiert. Nach einem fehlgeschlagenen `GET_VERSION` wird die Karte per `reselectCard()` (WUPA + Select) wieder ansprechbar gemacht. Eine gelesene Karte wird pro UID nur einmal eingelesen und dann gepuffert, damit die Anzeige nicht bei jedem Poll neu aufgebaut wird.

Kopieren (Dump/Restore)

`dumpCardToSd()` schreibt den Karteninhalt als Textdatei nach `/NFC-DUMPS/<uid>.nfc`. Bei MIFARE Classic wird pro Sektor ein **Schlüsselwörterbuch** (13 gängige Schlüssel) durchprobiert; der gefundene Schlüssel wird als Kommentar notiert und in die Trailer-Zeile eingesetzt (da Schlüssel A beim Lesen stets 00... liefert).

`restoreDumpToCard()` schreibt die Datenblöcke und – bei gültigen Zugriffsbits – auch die Sektor-Trailer zurück. Nicht geschrieben werden:

- **Block 0** (UID) — auf normalen Karten schreibgeschützt.
- **Schlüssel A** (Trailer Bytes 0-5) — prinzipiell nicht auslesbar.
- Trailer mit **ungültigen Zugriffsbits** — `validAccessBits()` prüft die Komplement-Kodierung, um ein dauerhaftes Sperren des Sektors zu verhindern.

Datenblöcke werden immer vor dem zugehörigen Trailer geschrieben, damit der Zugriff innerhalb des Sektors nicht verloren geht.

Bedienlogik

Tastenauswertung

`handleButtons()` wertet alle drei Tasten pro Durchlauf aus. Neben kurzem und langem Druck gibt es vier frei belegbare Aktionen (ShortcutAction: None, Reset, Reboot, UltiMenu, PowerOff):

- **B lang** (loslassen ohne zweite Taste)
- **C lang**
- **B lang, dann C** (Sequenz oder Akkord)
- **B lang, dann A**

Die Sequenz erlaubt Einhandbedienung: Nach dem Loslassen von B öffnet sich ein Zeitfenster (Kombi Zeit, 0,5–3,0 s), in dem A oder C die Kombination auslösen. Während B gehalten wird und im Wartefenster sind A und C von ihren Normalfunktionen abgekoppelt.

Im SD-Browser sind die Langdrücke abweichend belegt (A lang = Ordner zurück, B lang = Home, C halten = Auto-Scroll), damit Schnellschrollen nicht mit den Kürzeln kollidiert.

PowerOff-Absicherung

Zwei getrennte Bestätigungspfade:

- **POWER-Kachel:** zweiter Druck auf B innerhalb PowerOff Zeit.
- **Tastenkürzel:** je nach PowerOff Kombi direkt oder mit A-Bestätigung. Die Prüfung steht vor der B-Auswertung, damit die A-Taste, die „B lang + A“ auslöst, die eigene Abfrage nicht sofort selbst bestätigt.

Persistenz

Einstellungen liegen im NVS (Preferences, Namespace c64uremote). Zeitwerte werden in Zehntelsekunden gespeichert und beim Laden auf den gültigen Bereich begrenzt. `loadDefaultSettings()` stellt die Werkseinstellung her.

Die WLAN-Zugangsdaten liegen in einem **eigenen** Namensraum c64unet (siehe Kapitel *WLAN-Subsystem*) und bleiben deshalb auch nach *Factory Reset* erhalten. `build_env.h` liefert nur noch die Startwerte, solange dort nichts gespeichert ist.

Projektstruktur

M5Core_C64uRemote/	
├─ platformio.ini	Board, Bibliotheken, Partition, Upload
├─ README.md	
├─ LICENSE	MIT - Karl Prosser, Martin Oswald
├─ wifi.txt.example	Vorlage für /wifi.txt auf der SD-Karte
├─ .vscode/	empfohlene Erweiterungen, Editor-Einstellungen
├─ docs-src/	Markdown-Quellen der Handbücher
├─ doc/	fertige Handbücher (PDF)
├─ src/	deutsche Fassung
│ └─ main.cpp	gesamte Firmware
│ └─ build_env.h	Zugangsdaten (nicht versionieren)
│ └─ build_env.h.example	Vorlage
│ └─ 1MHz_logo_rgb565.h	Logo als RGB565-Array im Flash
└─ src-en/	
└─ main.cpp	englische Fassung, Code identisch

Fehlerdiagnose

Der serielle Monitor (115200 Baud) gibt beim Start Heap, RFID- und SD-Status aus. Beim Kartenlesen wird der gelesene Text protokolliert, bei abgelehnten Dateien Pfad und Endung, beim Disk-Mount die vollständige URL. Für die NFC-Analyse am Gerät dient NFC-Info Seite 2.

Typische Meldungen:

Meldung	Ursache
SET build_env.h	Zugangsdaten fehlen
NO WIFI	keine WLAN-Verbindung
AUTH? (Statusleiste)	Netzwerkpasswort im C64 gesetzt, hier nicht hinterlegt
TYP UNBEKANNT: ...	Dateiendung wird vom Ultimate nicht unterstützt
KARTE OHNE C64U-PFAD	Karte enthält keinen erkennbaren Pfad
LADEN DAUERT ZU LANGE	Disk-Autostart nach 3 min ohne READY

Quellen

- ReST API der Ultimate-Firmware: 1541u-documentation.readthedocs.io/en/latest/api/api_calls.html

- TeensyROM NFC Loader (Kartenformat): github.com/SensoriumEmbedded/TeensyROM/blob/main/doc
- Referenzimplementierung ultimate64 (Rust): github.com/mlund/ultimate64
- MFRC522_I2C-Bibliothek: github.com/kkloesener/MFRC522_I2C
- Originalprojekt C64uRemote: github.com/ReadyOS-C64/C64uRemote

Lizenz

C64uRemote steht unter der **MIT-Lizenz**. Der vollständige Lizenztext liegt als Datei LICENSE im Projektstamm.

Ursprung ist das Projekt **C64uRemote von Karl Prosser (@klumsy)**, <https://github.com/ReadyOS-C64/C64uRemote>, das er unter der MIT-Lizenz veröffentlicht hat. Diese Fassung ist eine daraus abgeleitete Erweiterung und steht unter denselben Bedingungen:

- Copyright (c) 2026 Karl Prosser – Originalprojekt
- Copyright (c) 2026 Martin Oswald (@mad, <https://1MHz.de>) – Portierung und Erweiterungen

Die MIT-Lizenz erlaubt es, die Software zu benutzen, zu verändern und weiterzugeben, auch kommerziell. Einzige Bedingung: **Copyright-Vermerk und Lizenztext müssen erhalten bleiben** und jeder Kopie beiliegen. Eine Gewährleistung oder Haftung ist ausgeschlossen.

Die eingebundenen Bibliotheken haben ihre eigenen Lizenzen: M5Unified und M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) sowie MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). Sie werden beim Bauen von PlatformIO geladen.

Entstehung

Portierung, Erweiterungen und Handbücher sind mit Unterstützung von Claude (Anthropic) entstanden. Konzept, Idee, Hardware-Entscheidungen und sämtliche Tests auf den echten Geräten: Martin Oswald (@mad).